

第 12 章 软件架构设计

传统的软件开发过程可以划分为从概念到实现的若干个阶段，包括软件计划、需求分析、软件设计、软件实现和软件测试等。在这种开发过程中，如何将需求分析的成果转换为软件设计，这个问题一直困扰着研究人员和实践工作者。近年来，软件工程界提出了各种需求工程和软件建模技术，然而，在软件需求和设计之间仍然存在一条很难逾越的鸿沟，从而很难有效地将需求转换为相应的设计。为此，学者们提出了软件架构（Software Architecture）的概念，并试图在软件需求与设计之间架起一座桥梁，重点解决系统结构和需求向实现平坦过渡的问题。

另一方面，随着软件系统规模越来越大、越来越复杂，整个系统的结构和规格说明显得越来越重要。在这种背景下，人们也逐渐认识到软件架构的重要性，促进了软件架构技术的快速发展和应用。

12.1 软件架构概述

软件架构为软件系统提供了一个结构、行为和属性的高级抽象，由构件的描述、构件的相互作用（连接件）、指导构件集成的模式以及这些模式的约束组成。软件架构指定了系统的组织结构和拓扑结构，并且显示了系统需求和构件之间的对应关系，提供了一些设计决策的基本原理。

软件架构虽脱胎于软件工程，但其形成也借鉴了计算机架构和网络架构中很多宝贵的思想和方法。近年来，软件架构已完全独立于软件工程，成为计算机科学的一个最新的研究方向和独立学科分支。软件架构研究的主要内容涉及软件架构描述、软件架构风格、软件架构评估和软件架构的形式化方法等。解决好软件的复用、质量和维护问题，是研究软件架构的根本目的。

1. 软件架构的意义

对于软件项目的开发来说，一个清晰的软件架构是首要的。鉴于架构的重要性，Perry 将软件架构视为软件开发中第一类重要的设计对象，Barry Boehm 也明确指出：“在没有设计出架构及其规则时，那么整个项目不能继续下去，而且架构应该看作是软件开发中可交付的中间产品”。由此可见，架构在软件开发中为不同的人员提供了共同的交流语言，体现并尝试了系统早期的设计决策，并作为系统设计的抽象，为实现框架和构件的共享和重用、基于架构的软件开发提供了有力的支持。

（1）架构是项目干系人进行交流的手段。软件架构代表了系统的高层抽象，不同的项目干系人关心着系统的不同方面，而这些方面都受架构的影响，因此，架构可能是所有项目干系人共同关心的一个重要因素。例如，用户关心系统是否满足可用性和可靠性需求；客户关心的是系统能否在规定时间内完成，并且开支在预算范围内；管理人员担心在经费支出和进度条件下，按此架构能否使开发团队成员在一定程度上独立开发，各部分的交互是否遵循统一的规范，开发进度是否可控；开发人员关心的是如何才能实现架构的各项目标。

(2) 架构是早期设计决策的体现。软件架构体现了系统最早的一组设计决策, 这些早期的约束比起以后的开发、设计、编码或运行及维护阶段的工作重要得多, 对系统生命周期的影响也大得多。早期决策的正确性最难以保证, 而且这些决策也最难以改变, 影响范围也最大。

(3) 架构明确了对系统实现的约束条件。所谓“实现”就是要用实体来显示出一个软件架构, 即要符合架构所描述的结构性的设计决策, 分割成规定的构件, 按规定方式互相交互。在具体实现时, 必须按照架构的设计, 将系统分成若干个组成部分, 各部分必须按照预定的方式进行交互, 而且每个部分也必须具有架构中所规定的外部特征。这些约束是在系统级或项目范围内作出的, 每个构件上工作的实现者是看不见的。这样一来, 可以分离关注点, 架构设计师不必是算法设计者或精通编程语言, 他们只需重点考虑系统的总体权衡 (tradeoff) 问题, 而构件的开发人员在架构给定的约束下进行开发。

(4) 架构决定了开发和维护组织的组织结构。架构包含了对系统的最高层次的分解, 因此一般被作为任务划分结构的基础。任务划分结构又规定了计划、调度及预算的单位, 决定了开发小组内部交流的渠道、配置控制和文件系统的组织、集成与测试计划和过程等。各开发小组按照架构中对各主要构件接口的规定进行交流。一旦进入维护阶段, 维护活动也会反映出软件架构, 常由不同的小组分别负责对各具体部分的维护。

(5) 架构制约着系统的质量属性。小的软件系统可以通过编程或调试措施来达到质量属性的要求, 而随着软件系统规模的扩大, 这种技巧也将越来越无法满足要求。因为在大型软件系统中, 质量属性更多的是由系统结构和功能划分来实现的, 而不再主要依靠所选用的算法或数据结构。可以使用对架构的评价来预测系统未来的质量属性, 架构评估技术可以对按某架构开发出来的软件产品的质量及缺陷做出比较准确的预测。

(6) 架构使推理和控制更改更简单。在整个软件生命周期内, 每个架构都将更改划分为三类, 分别是局部的、非局部的和架构级的变更。局部变更是最经常发生的, 也是最容易进行的, 只需修改某一个构件就可以实现。非局部变更的实现则需对多个构件进行修改, 但并不改动软件架构。架构级的变更是指会影响各部分的相互关系, 甚至要改动整个系统。所以, 一个优秀的架构应该能使更改简单易行。

(7) 架构有助于循序渐进的原型设计。一旦确定了架构, 就可以对它进行分析, 并将它按可执行模型来构造原型, 以减少项目开发中的潜在风险。

(8) 架构可以作为培训的基础。在对项目组新成员介绍所开发的系统时, 可以首先介绍系统的架构, 以及对构件之间如何交互从而实现系统需求的高层次的描述, 让项目新成员能很快进入角色。

(9) 架构是可传递和可复用的模型。软件架构体现了一个相对来说比较小又可理解的模型。软件架构级的复用意味着架构的决策能在具有相似需求的多个系统中发生影响, 这比代码级的复用要有更大的好处。通过对架构的抽象, 架构设计师能够对一些经过实践证明是非常有效的架构进行复用, 从而提高设计的效率和可靠性。

2. 软件架构的发展史

在软件系统规模迅速增大的同时, 软件开发方法也经历了一系列的变革。在此过程中, 软

件架构也由最初模糊的概念发展到一个渐趋成熟的理论和技术。

20 世纪 70 年代以前，尤其是在以 ALGOL 68 为代表的高级语言出现以前，软件开发基本上都是汇编程序设计，此阶段系统规模较小，很少明确考虑系统结构，一般不存在系统建模工作。20 世纪 70 年代中后期，由于结构化开发方法的出现与广泛应用，软件开发中出现了概要设计与详细设计，其主要任务是数据流设计与控制流设计，此时软件结构已作为一个明确的概念出现在系统开发中。

20 世纪 80 年代初到 90 年代中期，是 OO 方法兴起与成熟阶段。由于对象是数据与基于数据之上操作的封装。因此，在 OO 方法下，数据流设计与控制流设计统一为对象建模，同时，OO 方法还提出了一些其他的结构视图。例如，OMT 方法提出了功能视图、对象视图和动态视图，Booch 方法提出了类图、对象图、状态迁移图、交互图、模块图和进程图，UML 则从功能模型、静态模型、动态模型和配置模型等方面描述应用系统的结构。

20 世纪 90 年代以后，是基于构件的软件开发阶段。该阶段以过程为中心，强调软件开发采用构件化技术和架构技术，要求开发出的软件具备很强的自适应性、互操作性、可扩展性和可复用性。此阶段中，软件架构已经作为一个明确的文档和中间产品存在于软件开发过程中。同时，软件架构作为一门学科逐渐得到人们的重视，并成为软件工程领域的研究热点。

纵观软件架构技术的发展过程，从最初的无架构设计到现行的基于架构的软件开发，经历了 4 个阶段：

- (1) 无架构设计阶段。以汇编语言进行小规模应用程序开发为特征。
- (2) 萌芽阶段。出现了程序结构设计主题，以控制流图和数据流图构成软件结构为特征。
- (3) 初级阶段。出现了从不同侧面描述系统的结构模型，以 UML 为典型代表。

(4) 高级阶段。以描述系统的高层抽象结构为中心，不关心具体的建模细节，划分了架构模型与传统软件结构的界限。该阶段以 Kruchten 提出的“4+1”模型为标志。有关该模型的详细知识将在 12.2 节中介绍。

12.2 软件架构建模

软件架构设计的首要问题是如何表示软件架构，即如何对软件架构建模。根据建模的侧重点不同，可以把软件架构的模型分为 5 种，分别是结构模型、框架模型、动态模型、过程模型和功能模型。

(1) 结构模型。这是一种最直观和最普遍的建模方法，它以构件、连接件和其他概念来刻画架构，并力图通过架构来反映系统的重要语义内容，包括系统的配置、约束、隐含的假设条件、风格和性质等。研究结构模型的核心是架构描述语言。

(2) 框架模型。框架模型与结构模型类似，但它不太侧重描述结构的细节而更侧重于整体结构。框架模型主要以一些特殊的问题为目标建立只针对和适应该问题的架构。

(3) 动态模型。动态模型是对结构模型或框架模型的补充，研究系统的粗粒度行为性质。例如，描述系统的重新配置或演化等，这类系统通常是激励型的。

(4) 过程模型。过程模型研究构建系统的步骤和过程。

(5) 功能模型。功能模型认为架构是由一组功能构件按层次组成的，下层向上层提供服务。功能模型可以看作是一种特殊的框架模型。

在上述5种模型中，最常用的是结构模型和动态模型。这5种模型各有所长，将它们有机地统一在一起，形成一个完整的模型来刻画软件架构更合适。例如，Kruchten在1995年提出了一个“4+1”的视图模型。“4+1”视图模型从5个不同的视角来描述软件架构，每个视图只关心系统的一个侧面，5个视图结合在一起才能反映软件架构的全部内容。“4+1”视图模型如图12-1所示。

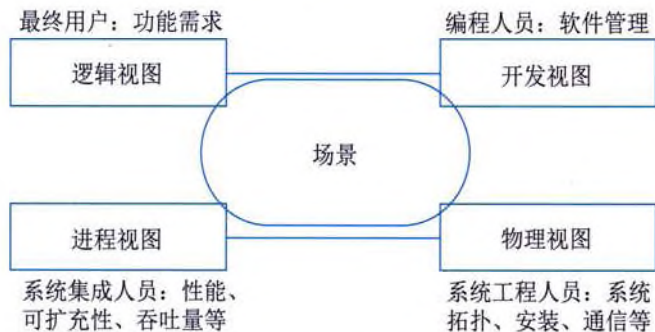


图 12-1 “4+1” 视图模型

(1) 逻辑视图。逻辑视图主要支持系统的功能需求，即系统提供给最终用户的服务。在逻辑视图中，系统分解成一系列的功能抽象，这些抽象主要来自问题领域。这种分解不但可以用来进行功能分析，而且可用作标识在整个系统的各个不同部分的通用机制和设计元素。在OO技术中，通过抽象、封装和继承，可以用对象模型来代表逻辑视图，用类图来描述逻辑视图。逻辑视图中使用的风格为面向对象的风格，在设计中要注意保持一个单一的、内聚的对象模型贯穿整个系统。

(2) 开发视图。开发视图也称为模块视图，在UML中被称为实现视图，它主要侧重于软件模块的组织和管理。开发视图要考虑软件内部的需求，例如，软件开发的容易性、软件复用和软件的通用性，要充分考虑到由于具体开发工具的不同而带来的局限性。开发视图通过系统I/O关系的模型图和子系统图来描述。

(3) 进程视图。进程视图侧重于系统的运行特性，主要关注一些非功能性需求，例如，系统的性能和可用性等。进程视图强调并发性、分布性、系统集成性和容错能力，以及逻辑视图中的功能抽象如何适合进程结构等，它也定义了逻辑视图中的各个类的操作具体是在哪一个线程中被执行的。进程视图可以描述成多层抽象，每个级别分别关注不同的方面。

(4) 物理视图。物理视图在UML中被称为部署视图，主要考虑如何把软件映射到硬件上，它通常要考虑到解决系统拓扑结构、系统安装和通信等问题。当软件运行于不同的物理节点上时，各视图中的构件都直接或间接地对应于系统的不同节点上。因此，从软件到节点的映射要有较高的灵活性，当环境改变时，对系统其他视图的影响最小化。

(5) 场景。场景可以看作是那些重要系统活动的抽象，它使4个视图有机联系起来，从某种意义上说场景是最重要的需求抽象。场景视图对应UML中的用例视图。在开发软件架构时，它可以帮助架构设计师找到构件及其相互关系。同时，架构设计师也可以用场景来分析一个特

定的视图，或描述不同视图的构件之间是如何相互作用的。场景可以用文本表示，也可以用图形表示。例如，图 12-2 是一个小型电话呼叫系统场景片段的图形描述，相应的文本表示如下：

① 小王的电话控制器检测和验证电话从挂机到摘机状态的转变，且发送一个消息以唤醒相应的终端对象。

② 终端分配一定的资源，且通知控制器发出某种拨号音。

③ 控制器接收所拨号码并传给终端。

④ 终端使用编号计划分析号码。

⑤ 当一个有效的拨号序列进入时，终端打开一个会话。

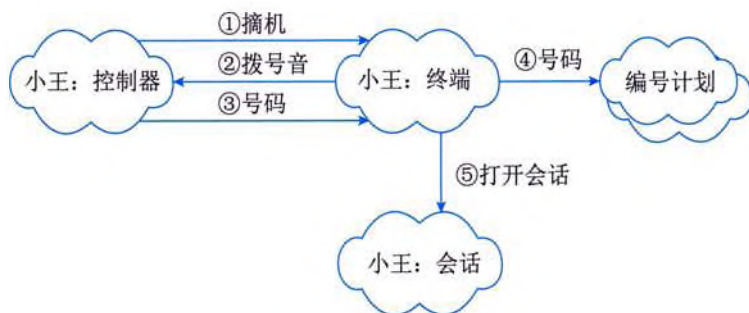


图 12-2 本地呼叫场景的一个原型

从以上分析可知，逻辑视图和开发视图描述系统的静态结构，而进程视图和物理视图描述系统的动态结构。对于不同的软件系统来说，侧重角度也有所不同。例如，对于 MIS 来说，比较侧重于从逻辑视图和开发视图来描述系统；而对于实时控制系统来说，则比较注重于从进程视图和物理视图来描述系统。

12.3 软件架构风格

软件架构，即软件体系结构。软件体系结构设计的一个核心目标是重复的体系结构模式，即达到体系结构级的软件重用。也就是说，在不同的软件系统中，使用同一体系结构。基于这个目标，主要任务是研究和实践软件体系结构风格和类型问题。

12.3.1 软件架构风格概述

软件体系结构风格是描述某一特定应用领域中系统组织方式的惯用模式。体系结构风格定义一个系统家族，即一个体系结构定义一个词汇表和一组约束。词汇表中包含一些构件和连接件类型，而这组约束指出系统是如何将这些构件和连接件组合起来的。体系结构风格反映了领域中众多系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。对软件体系结构风格的研究和实践促进对设计重用，一些经过实践证实的解决方案也可以可靠地用于解决新的问题。例如，如果某人把系统描述为“客户/服务器”模式，则不必给出设计细节，我们立刻就会明白系统是如何组织和工作的。

12.3.2 数据流体系结构风格

数据流体系结构是一种计算机体系结构，直接与传统的冯·诺依曼体系结构或控制流体系结构进行了对比。数据流体系结构没有概念上的程序计数器：指令的可执行性和执行仅基于指令输入参数的可用性来确定，因此，指令执行的顺序是不可预测的，即行为是不确定的。数据流体系结构风格主要包括批处理风格和管道-过滤器风格。

1. 批处理体系结构风格

在批处理风格（如图12-3所示）的软件体系结构中，每个处理步骤是一个单独的程序，每一步必须在前一步结束后才能开始，并且数据必须是完整的，以整体的方式传递。它的基本构件是独立的应用程序，连接件是某种类型的媒介。连接件定义了相应的数据流图，表达拓扑结构。

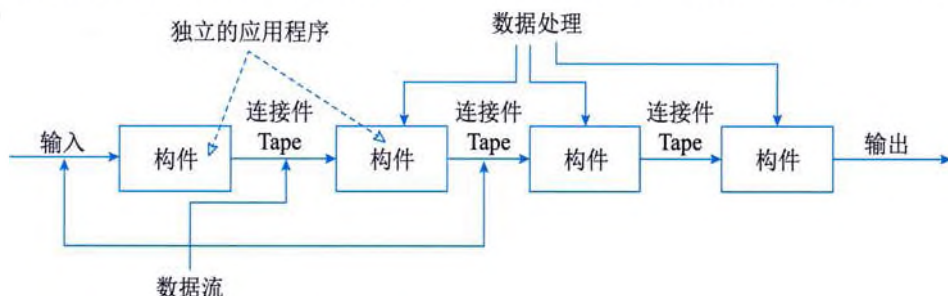


图 12-3 批处理体系结构风格

2. 管道-过滤器体系结构风格

当数据源源不断地产生，系统就需要对这些数据进行若干处理（分析、计算、转换等）。现有的解决方案是把系统分解为几个序贯的处理步骤，这些步骤之间通过数据流连接，一个步骤的输出是另一个步骤的输入。每个处理步骤由一个过滤器（Filter）实现，处理步骤之间的数据传输由管道（Pipe）负责。每个处理步骤（过滤器）都有一组输入和输出，过滤器从管道中读取输入的数据流，经过内部处理，然后产生输出数据流并写入管道中。因此，管道-过滤器体系结构风格（如图12-4所示）的基本构件是过滤器，连接件是数据流传输管道，将一个过滤器的输出传到另一过滤器的输入。

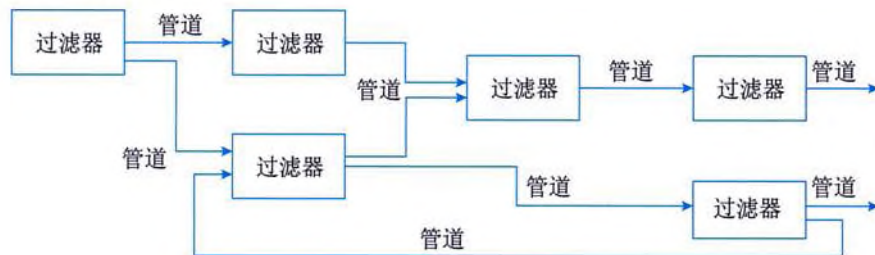


图 12-4 管道-过滤器体系结构风格

12.3.3 调用/返回体系结构风格

调用返回体系结构风格是指在系统中采用了调用与返回机制。利用调用-返回实际上是一

种分而治之的策略，其主要思想是将一个复杂的大系统分解为若干子系统，以便降低复杂度，并且增加可修改性。程序从其执行起点开始执行该构件的代码，程序执行结束，将控制返回给程序调用构件。调用/返回体系结构风格主要包括主程序/子程序风格、面向对象风格、层次型风格以及客户端/服务器风格。

1. 主程序/子程序风格

主程序/子程序风格一般采用单线程控制，把问题划分为若干处理步骤，构件即为主程序和子程序。子程序通常可合成为模块。过程调用作为交互机制，即充当连接件。调用关系具有层次性，其语义逻辑表现为子程序的正确性取决于它调用的子程序的正确性。

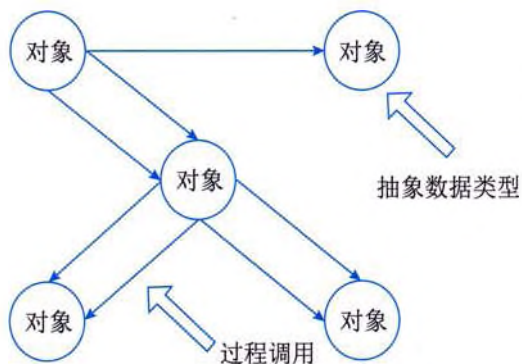


图 12-5 面向对象体系结构风格

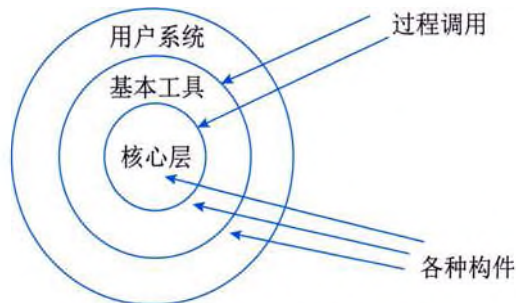


图 12-6 层次型体系结构风格

2. 面向对象体系结构风格

抽象数据类型概念对软件系统有着重要作用，目前软件界已普遍转向使用面向对象系统。这种风格建立在数据抽象和面向对象的基础上，数据的表示方法和它们的相应操作封装在一个抽象数据类型或对象中。这种风格的构件是对象，或者说是抽象数据类型的实例（如图 12-5 所示）。

3. 层次型体系结构风格

层次系统（如图 12-6 所示）的组成为一个层次结构，每一层为上层提供服务，并作为下层的客户。在一些层次系统中，除了一些精心挑选的输出函数外，内部的层接口只对相邻的层可见。在这样的系统中，构件在层上实现了虚拟机。连接件由决定层间如何交互的协议来定义，拓扑约束包括对相邻层间交互的约束。由于每一层最多只影响两层，同时只要给相邻层提供相同的接口，允许每层用不同的方法实现，这同样为软件重用提供了强大的支持。

4. 客户端/服务器体系结构风格

在 IT 发展过程中，网络计算经历了从集中式计算模型到分布式计算模型的演变。在集中式计算技术时代，广泛使用的是大型机（或小型机）计算模型。它是通过一台物理上与宿主机相连接的非智能终端来实现宿主机上的应用程序。在多用户环境中，宿主机应用程序既负责与用户的交互，又负责对数据的管理。集中式的系统使用户能共享贵重的硬件设备，例如，磁盘机、打印机和调制解调器等。但随着用户的增多，对宿主机能力的要求增高，而且开发人员必须为每个新的应用重新设计同样的数据管理构件。

20 世纪 80 年代以后，集中式结构逐渐被以 PC 为主的微机网络所取代。PC 和工作站的采

(1) 开发成本较高。C/S 架构对客户端软硬件配置要求较高，尤其是软件的不不断升级，对硬件要求不断提高，增加了整个系统的成本。

(2) 客户端程序设计复杂。采用 C/S 架构进行软件开发，大部分工作量放在客户端的程序设计上，客户端显得十分庞大。

(3) 用户界面风格不一，使用繁杂，不利于推广使用。

(4) 软件移植困难。采用不同开发工具或平台开发的软件，一般互不兼容，不能或很难移植到其他平台上运行。

(5) 软件维护和升级困难。采用 C/S 架构的软件要升级，开发人员必须到现场为客户机升级，每个客户机上的软件都需要维护。对软件的一个小小改动，每一个客户端都必须更新。

(6) 新技术不能轻易应用。因为一个软件平台及开发工具一旦选定，不可能轻易更改。

(7) 可扩展性差。C/S 架构是单一服务器且以局域网为中心的，所以难以扩展至大型企业广域网或 Internet，软硬件的组合和集成能力有限。客户机的负荷太重，难以管理大量的客户机，系统的性能容易变坏。

(8) 系统安全性难以保证。因为客户端程序可以直接访问数据库服务器，所以，在客户端计算机上的其他程序也可想办法访问数据库服务器，从而使数据库的安全性受到威胁。

正是因为 C/S 架构有这么多缺点，因此，三层 C/S 架构应运而生。为了区分，把传统的 C/S 架构称为二层 C/S 架构。

2) 三层 C/S 架构

与二层 C/S 架构相比，在三层 C/S 架构中，增加了一个应用服务器。可以将整个应用逻辑驻留在应用服务器上，而只有表示层存在于客户机上。这种客户机称为瘦客户机 (thin client)。三层 C/S 架构将应用系统分成表示层、功能层和数据层三个部分，如图 12-8 所示。

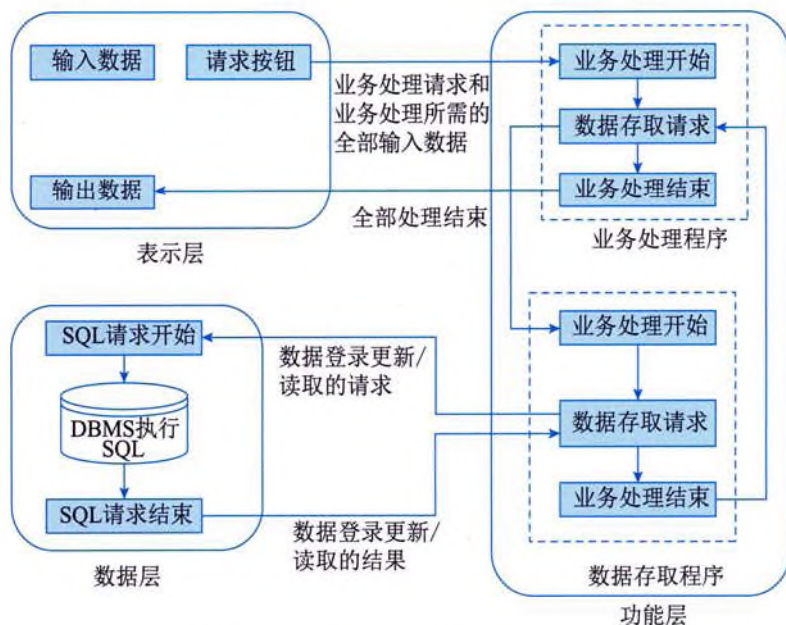


图 12-8 三层 C/S 架构的一般处理流程

(1) 表示层。表示层是系统的用户接口部分, 担负着用户与系统之间的对话功能。它用于检查用户从键盘等输入的数据, 显示输出的数据。为使用户能直观地进行操作, 通常使用图形用户界面, 操作简单、易学易用。在变更用户界面时, 只需改写显示控制和数据检查程序, 而不影响其他两层。检查的内容也只限于数据的形式和取值的范围, 不包括有关业务本身的处理逻辑。

(2) 功能层。功能层也称为业务逻辑层, 是将具体的业务处理逻辑编入程序中。例如, 在制作订购合同时计算合同金额、按照预定的格式配置数据、打印订购合同, 而处理所需的数据则要从表示层或数据层取得。

(3) 数据层。数据层相当于二层 C/S 架构中的服务器, 负责对 DBMS 进行管理和控制。

三层 C/S 架构对这三层进行明确分割, 并在逻辑上使其独立。在二层 C/S 架构中, 数据层作为 DBMS 已经独立出来, 所以, 三层 C/S 架构的关键是要将表示层和功能层分离成各自独立的程序, 并且还要使这两层间的接口简洁明了。通常的做法是只将表示层配置在客户机中, 如图 12-9 (a) 或图 12-9 (b) 所示。如果像图 12-9 (c) 所示的那样连功能层也放在客户机中, 与二层 C/S 架构相比, 其程序的可维护性要好得多, 但是其他问题并未得到解决。客户机的负荷太重, 其业务处理所需的数据要从服务器传给客户机, 所以系统的性能容易变坏。

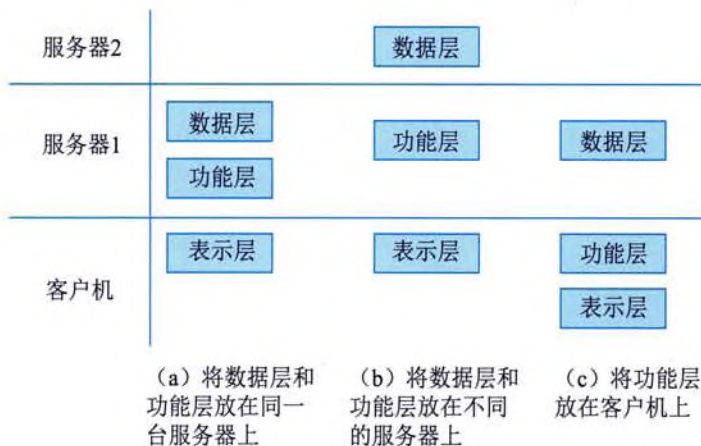


图 12-9 三层 C/S 架构的物理部署

如果将功能层和数据层分别放在不同的服务器中, 如图 12-9 (b) 所示, 则服务器之间也要进行数据传送。由于三层是分别放在各自不同的硬件系统上的, 所以灵活性很高, 能够适应客户机数目的增加和处理负荷的变动。例如, 在追加新业务处理时, 可以相应增加装载功能层的服务器 (应用服务器)。因此, 系统规模越大, 这种形态的优点就越显著。在三层 C/S 架构中, 中间件是最重要的构件。

与传统的二层 C/S 架构相比, 三层 C/S 架构具有以下优点:

(1) 允许合理地划分三层的功能, 使之在逻辑上保持相对独立性, 从而使整个系统的逻辑结构更为清晰, 能提高系统的可维护性和可扩展性。

(2) 允许更灵活、有效地选用相应的平台和硬件系统, 使之在处理负荷能力上与处理特性上分别适应于结构清晰的三层, 并且这些平台和各个组成部分可以具有良好的可升级性和开放性。

(3) 系统的各层可以并行开发, 各层也可以选择各自最适合的开发语言, 使之能并行且高

效地进行开发，达到较高的性能价格比。对每一层的处理逻辑的开发和维护也会更容易些。

(4) 利用功能层可以有效地隔离表示层与数据层，未授权的用户难以绕过功能层而利用数据库工具或黑客手段非法访问数据层，这就为严格的安全管理奠定了坚实的基础。

但是，若三层 C/S 架构各层间的通信效率不高，即使分配给各层的硬件能力很强，其作为整体来说也达不到所要求的性能。此外，设计时必须慎重考虑三层间的通信方法、通信频度和数据量，这是三层 C/S 架构设计的关键问题。

3) B/S 架构

浏览器 / 服务器 (Browser/Server, B/S) 架构是三层 C/S 架构的一种实现方式，其具体结构为“浏览器 / Web 服务器 / 数据库服务器”。B/S 架构利用不断成熟的 WWW 浏览器技术，结合浏览器的多种脚本语言，用通用浏览器就实现了原来需要复杂的专用软件才能实现的强大功能，并节约了开发成本。从某种程度上来说，B/S 架构是一种全新的软件架构。

在 B/S 架构中，除了数据库服务器外，应用程序以网页形式存放于 Web 服务器上，用户运行某个应用程序时，只需在客户端的浏览器中键入相应的网址，调用 Web 服务器上的应用程序，并对数据库进行操作，完成相应的数据处理工作，最后将结果通过浏览器显示给用户。基于 B/S 架构的软件，系统安装、修改和维护全在服务器端解决。用户在使用系统时，仅仅需要一个浏览器就可运行全部的模块，真正达到了“零客户端”的功能，很容易在运行时自动升级。

但是，与 C/S 架构相比，B/S 架构也有许多不足之处，例如，缺乏对动态页面的支持能力，没有集成有效的数据库处理功能；安全性难以控制；采用 B/S 架构的系统在数据查询等响应速度上远远低于 C/S 架构；B/S 架构的数据提交一般以页面为单位，数据的动态交互性不强，不利于 OLTP 应用。

12.3.4 以数据为中心的体系结构风格

以数据为中心的体系结构风格主要包括仓库风格和黑板风格。

1. 仓库体系结构风格

仓库 (repository) 是存储和维护数据的中心场所。在仓库体系结构风格 (如图 12-10 所示) 中，有两种不同的构件：中央数据结构 (说明当前数据的状态)，以及一组对中央数据进行操作的独立构件，仓库与独立构件间的相互作用在系统中会有大的变化。这种风格的连接件即为仓库与独立构件之间的交互。



图 12-10 仓库体系结构风格

2. 黑板体系结构风格

黑板体系结构风格 (如图 12-11 所示) 适用于解决复杂的非结构化问题，能在求解过程中

2. 规则系统体系结构风格

基于规则的系统（如图 12-13 所示）包括规则集、规则解释器、规则 / 数据选择器及工作内存。

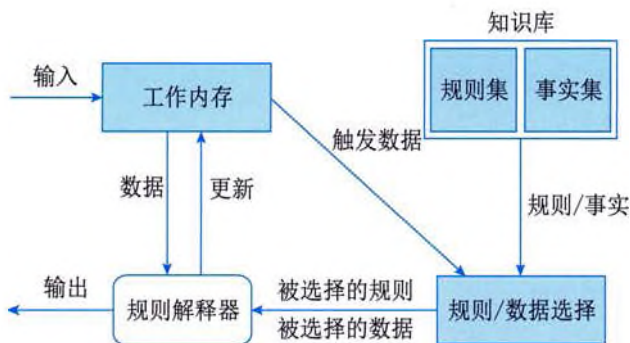


图 12-13 规则系统体系结构风格

12.3.6 独立构件体系结构风格

独立构件风格主要强调系统中的每个构件都是相对独立的个体，它们之间不直接通信，以降低耦合度，提升灵活性。独立构件风格主要包括进程通信和事件系统风格。

1. 进程通信体系结构风格

在进程通信结构体系结构风格中，构件是独立的过程，连接件是消息传递。这种风格的特点是：构件通常是命名过程，消息传递的方式可以是点到点、异步或同步方式及远程过程调用等。

2. 事件系统体系结构风格

事件系统风格（如图 12-14 所示）基于事件的隐式调用风格的思想，构件不直接调用一个过程，而是触发或广播一个或多个事件。系统中其他构件中的过程在一个或多个事件中注册，当一个事件被触发，系统自动调用在这个事件中注册的所有过程，这样，一个事件的触发就导致了另一模块中的过程的调用。

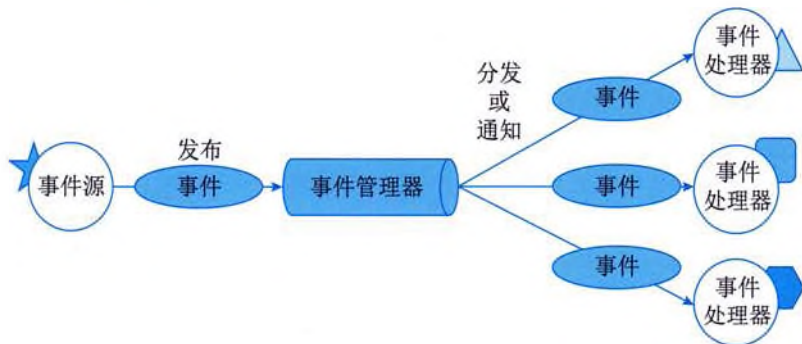


图 12-14 事件系统体系结构风格

从架构上说,这种风格的构件是一些模块,这些模块既可以是一些过程,又可以是一些事件的集合。过程可以用通用的方式调用,也可以在系统事件中注册一些过程,当发生这些事件时,过程被调用。

基于事件的隐式调用风格的主要特点是事件的触发者并不知道哪些构件会被这些事件影响。这使得不能假定构件的处理顺序,甚至不知道哪些过程会被调用,因此,许多隐式调用的系统也包含显式调用作为构件交互的补充形式。

支持基于事件的隐式调用的应用系统很多。例如,在编程环境中用于集成各种工具,在数据库管理系统中确保数据的一致性约束,在用户界面系统中管理数据,以及在编辑器中支持语法检查。例如在某系统中,编辑器和变量监视器可以登记相应 Debugger 的断点事件。当 Debugger 在断点处停下时,它声明该事件,由系统自动调用处理程序,如编辑器可以滚屏(返回)到断点,变量监视器刷新变量数值。而 Debugger 本身只声明事件,并不关心哪些过程会启动,也不关心这些过程做什么处理。

12.3.7 面向服务的架构风格

迄今为止,对于面向服务的架构(Service-Oriented Architecture, SOA)还没有一个公认的定义。许多组织从不同的角度和不同的侧面对 SOA 进行了描述,较为典型的有以下三个:

(1) W3C 的定义: SOA 是一种应用程序架构,在这种架构中,所有功能都定义为独立的服务,这些服务带有定义明确的可调用接口,能够以定义好的顺序调用这些服务来形成业务流程。

(2) Service-architecture.com 的定义: 服务是精确定义、封装完善、独立于其他服务所处环境和状态的函数。SOA 本质上是服务的集合,服务之间彼此通信,这种通信可能是简单的数据传送,也可能是两个或更多的服务协调进行某些活动。服务之间需要某些方法进行连接。

(3) Gartner 的定义: SOA 是一种 C/S 架构的软件设计方法,应用由服务和使用者组成,SOA 与大多数通用的 C/S 架构模型不同之处,在于它着重强调构件的松散耦合,并使用独立的标准接口。

1. SOA 概述

SOA 是一种在计算环境中设计、开发、部署和管理离散逻辑单元(服务)模型的方法。SOA 并不是一个新鲜事物,而只是面向对象模型的一种替代。虽然基于 SOA 的系统并不排除使用 OOD 来构建单个服务,但是其整体设计却是面向服务的。由于 SOA 考虑到了系统内的对象,所以虽然 SOA 是基于对象的,但是作为一个整体,它却不是面向对象的。

SOA 系统原型的一个典型例子是 CORBA,它已经出现很长时间,其定义的概念与 SOA 相似。SOA 建立在 XML 等新技术的基础上,通过使用基于 XML 的语言来描述接口,服务已经转到更动态且更灵活的接口系统中, CORBA 中的 IDL 无法与之相比。图 12-15 描述了一个完整的 SOA 模型。

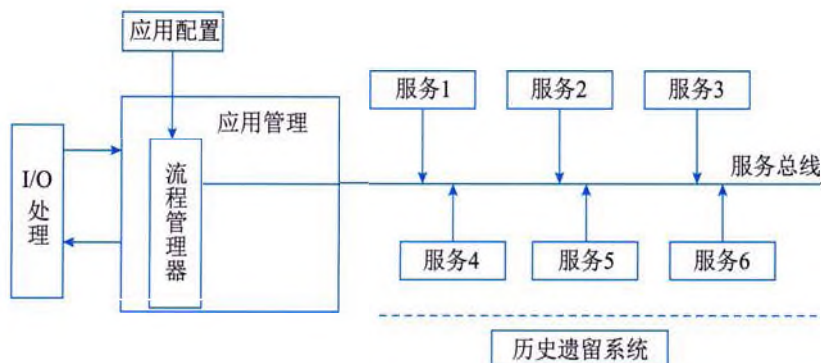


图 12-15 SOA 模型示例

在 SOA 模型中，所有的功能都定义成了独立的服务。服务之间通过交互和协调完成业务的整体逻辑。所有的服务通过服务总线或流程管理器来连接。这种松散耦合的架构使得各服务在交互过程中无须考虑双方的内部实现细节，以及部署在什么平台上。

1) 服务的基本结构

一个独立的服务基本结构如图 12-16 所示。

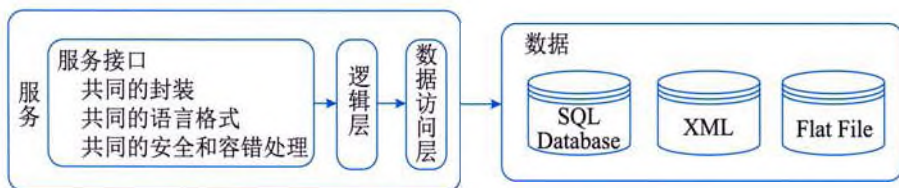


图 12-16 单个服务内部结构

从图 12-16 可以看出，服务模型的表示层从逻辑层分离出来，中间增加了服务对外的接口层。通过服务接口的标准化描述，使得服务可以提供给任何异构平台和任何用户接口使用。这允许并支持基于服务的系统成为松散耦合、面向构件和跨技术实现的，服务请求者很可能根本不知道服务在哪里运行、是由哪种语言编写，以及消息的传输路径，而是只需要提出服务请求，然后就会得到答案。

2) SOA 设计原则

SOA 架构继承了来自对象和构件设计的各种原则，例如，封装和自我包含等。那些保证服务的灵活性、松散耦合和复用能力的设计原则，对 SOA 架构来说同样是非常重要的。关于服务，一些常见的设计原则如下：

(1) 明确定义的接口。服务请求者依赖于服务规约来调用服务，因此，服务定义必须长时间稳定，一旦公布，不能随意更改；服务的定义应尽可能明确，减少请求者的不适当使用；不要让请求者看到服务内部的私有数据。

(2) 自包含和模块化。服务封装了那些在业务上稳定、重复出现的活动和构件，实现服务的功能实体是完全独立自主的，独立进行部署、版本控制、自我管理和恢复。

(3) 粗粒度。服务数量不应该太多, 依靠消息交互而不是远程过程调用, 通常消息量比较大, 但是服务之间的交互频度较低。

(4) 松耦合。服务请求者可见的是服务的接口, 其位置、实现技术、当前状态和私有数据等, 对服务请求者而言是不可见的。

(5) 互操作性、兼容和策略声明。为了确保服务规约的全面和明确, 策略成为一个越来越重要的方面。这可以是与技术相关的内容, 例如, 一个服务对安全性方面的要求; 也可以是与业务有关的语义方面的内容, 例如, 需要满足的费用或者服务级别方面的要求, 这些策略服务的交互是非常重要的。

3) 服务构件与传统构件

服务构件架构 (Service Component Architecture, SCA) 是基于 SOA 思想描述服务之间组合和协作的规范, 它用于描述使用 SOA 构建应用程序和系统的模型。它可简化使用 SOA 进行的应用程序开发和实现工作。SCA 提供了构建粗粒度构件的机制, 这些粗粒度构件由细粒度构件组装而成。SCA 将传统中间件编程从业务逻辑分离出来, 从而使程序员免受其复杂性的困扰。它允许开发人员集中精力编写业务逻辑, 而不必将大量的时间花费在更为底层的技术实现上。

SCA 服务构件与传统构件的主要区别在于, 服务构件往往是粗粒度的, 而传统构件以细粒度居多; 服务构件的接口是标准的, 主要是服务描述语言接口, 而传统构件常以具体 API 的形式出现; 服务构件的实现与语言是无关的, 而传统构件常绑定某种特定的语言; 服务构件可以通过构件容器提供 QoS 的服务, 而传统构件完全由程序代码直接控制。

2. SOA 的关键技术

SOA 伴随着无处不在的标准, 为企业的现有资产或投资带来了更好的复用性。SOA 能够在最新的和现有的系统之上创建应用, 借助现有的应用产生新的服务, 为企业提供更好的灵活性来构建系统和业务流程。SOA 是一种全新的架构, 为了支持其各种特性, 相关的技术规范不断推出。与 SOA 紧密相关的技术主要有统一描述、发现和集成 (Universal Description Discovery and Integration, UDDI)、Web 服务描述语言 (Web Service Description Language, WSDL)、简单对象访问协议 (Simple Object Access Protocol, SOAP) 和表述性状态转移 (Representational State Transfer, REST) 等, 而这些技术都是以 XML 为基础发展起来的。

1) UDDI

UDDI 提供了一种服务发布、查找和定位的方法, 是服务的信息注册规范, 以便被需要该服务的用户发现和使用它。UDDI 规范描述了服务的概念, 同时也定义了一种编程接口。通过 UDDI 提供的标准接口, 企业可以发布自己的服务供其他企业查询和调用, 也可以查询特定服务的描述信息, 并动态绑定到该服务上。

在 UDDI 技术规范中, 主要包含以下三部分内容:

(1) 数据模型。UDDI 数据模型是一个用于描述业务组织和服务的 XML Schema。

(2) API。UDDI API 是一组用于查找或发布 UDDI 数据的方法, UDDI API 基于 SOAP。

(3) 注册服务。UDDI 注册服务是 SOA 中的一种基础设施, 对应着服务注册中心的角色。

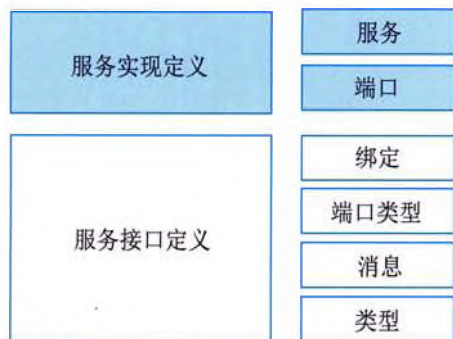


图 12-17 基本服务描述

2) WSDL

WSDL 是对服务进行描述的语言，它有一套基于 XML 的语法定义。WSDL 描述的重点是服务，它包含服务实现定义和服务接口定义，如图 12-17 所示。

采用抽象接口定义对于提高系统的扩展性很有帮助。服务接口定义就是一种抽象的、可重用的定义，行业标准组织可以使用这种抽象的定义来规定一些标准的服务类型，服务实现者可以根据这些标准定义来实现具体的服务。

服务实现定义描述了给定服务提供者如何实现特定的服务接口。服务实现定义中包含对服务和端口的描述。一个服务往往会包含多个服务访问入口，而每个访问入口都会使用一个端口元素来描述。端口描述的是一个服务访问入口的部署细节，例如，通过哪个地址来访问，应当使用怎样的消息调用模式来访问等。

3) SOAP

SOAP 定义了服务请求者和服务提供者之间的消息传输规范。SOAP 用 XML 来格式化消息，用 HTTP 来承载消息。通过 SOAP，应用程序可以在网络中进行数据交换和远程过程调用（Remote Procedure Call，RPC）。SOAP 主要包括以下 4 个部分：

（1）封装。SOAP 封装定义了一个整体框架，用来表示消息中包含什么内容，谁来处理这些内容，以及这些内容是可选的还是必须的。

（2）编码规则。SOAP 编码规则定义了一种序列化的机制，用于交换系统所定义的数据类型的实例。

（3）RPC 表示。SOAP RPC 表示定义了一个用来表示远程过程调用和应答的协议。

（4）绑定。SOAP 绑定定义了一个使用底层传输协议来完成在节点之间交换 SOAP 封装的约定。

SOAP 消息基本上是从发送端到接收端的单向传输，但它们常常结合起来执行类似于请求/应答的模式。所有的 SOAP 消息都使用 XML 进行编码。SOAP 消息包括以下三个部分：

（1）封装（信封）。封装的元素名是 Envelope，在表示消息的 XML 文档中，封装是顶层元素，在 SOAP 消息中必须出现。

（2）SOAP 头。SOAP 头的元素名是 Header，提供了向 SOAP 消息中添加关于这条 SOAP 消息的某些要素的机制。SOAP 定义了少量的属性用来表明这项要素是否可选以及由谁来处理。SOAP 头在 SOAP 消息中可能出现，也可能不出现。如果出现的话，必须是 SOAP 封装元素的第一个直接子元素。

（3）SOAP 体。SOAP 体的元素名是 Body，是包含消息的最终接收者想要的信息的容器。SOAP 体在 SOAP 消息中必须出现且必须是 SOAP 封装元素的直接子元素。如果有头元素，则 SOAP 体必须直接跟在 SOAP 头元素之后；如果没有头元素，则 SOAP 体必须是 SOAP 封装元素的第一个直接子元素。

4) REST

REST 是一种只使用 HTTP 和 XML 进行基于 Web 通信的技术,可以降低开发的复杂性,提高系统的可伸缩性。它的简单性和缺少严格配置文件特性,使它与 SOAP 很好地隔离开。REST 从根本上来说只支持几个操作 (POST、GET、PUT 和 DELETE),这些操作适用于所有的消息。REST 提出了如下一些设计概念和准则:

- (1) 网络上的所有事物都被抽象为资源。
- (2) 每个资源对应一个唯一的资源标识。
- (3) 通过通用的连接件接口对资源进行操作。
- (4) 对资源的各种操作不会改变资源标识。
- (5) 所有的操作都是无状态的。

3. SOA 的实现方法

SOA 只是一种概念和思想,需要借助于具体的技术和方法来实现它。从本质上来看,SOA 是用本地计算模型来实现一个分布式的计算应用,也有人称这种方法为“本地化设计,分布式工作”模型。CORBA、DCOM 和 EJB 等都属于这种解决方式,也就是说,SOA 最终可以基于这些标准来实现。另外,这些标准分别使用的 ORB、RPC 和远程方法调用 (Remote Method Invocation, RMI) 等技术,感兴趣的同学可进一步阅读其他书籍学习,此处不再赘述。

从逻辑上和高层抽象来看,目前,实现 SOA 的方法也比较多,其中主流方式有 Web Service、企业服务总线和服务注册表。

1) Web Service

在 Web Service (Web 服务) 的解决方案中,一共有三种工作角色,其中服务提供者和服务请求者是必须的,服务注册中心是一个可选的角色。它们之间的交互和操作构成了 SOA 的一种实现架构,如图 12-18 所示。

(1) 服务提供者。服务提供者是服务的所有者,该角色负责定义并实现服务,使用 WSDL 对服务进行详细、准确、规范的描述,并将该描述发布到服务注册中心,供服务请求者查找并绑定使用。

(2) 服务请求者。服务请求者是服务的使用者,虽然服务面向的是程序,但程序的最终使用者仍然是用户。从架构的角度看,服务请求者是查找、绑定并调用服务,或服务进行交互的应用程序。服务请求者角色可以由浏览器来担当,由人或程序 (例如,另外一个服务) 来控制。

(3) 服务注册中心。服务注册中心是连接服务提供者和服务请求者的纽带,服务提供者在此发布他们的服务描述,而服务请求者在服务注册中心查找他们需要的服务。不过,在某些情况下,服务注册中心是整个模型中的可选角色。例如,如果使用静态绑定的服务,则服务提供者可以把描述直接发送给服务请求者。

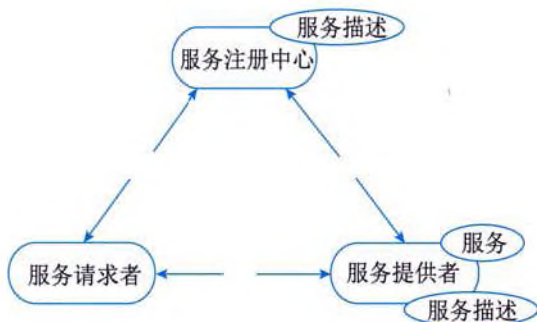


图 12-18 Web Service 模型

Web Service 模型中的操作包括发布、查找和绑定等,可以单次或反复出现。

(1) 发布。为了使用户能够访问服务,服务提供者需要发布服务描述,以便服务请求者可以查找它。

(2) 查找。在查找操作中,服务请求者直接检索服务描述或在服务注册中心查询所要求的服务类型。对服务请求者而言,可能会在生命周期的两个不同阶段中涉及查找操作,首先是在设计阶段,为了程序开发而查找服务的接口描述;其次是在运行阶段,为了调用而查找服务的位置描述。

(3) 绑定。在绑定操作中,服务请求者使用服务描述中的绑定细节来定位、联系并调用服务,从而在运行时与服务进行交互。绑定可以分为动态绑定和静态绑定。在动态绑定中,服务请求者通过服务注册中心查找服务描述,并动态地与服务交互;在静态绑定中,服务请求者已经与服务提供者达成默契,通过本地文件或其他方式直接与服务进行绑定。

在采用 Web Service 作为 SOA 的实现技术时,应用系统大致可以分为如下 6 个层次:底层传输层、服务通信协议层、服务描述层、服务层、业务流程层和服务注册层。

(1) 底层传输层。底层传输层主要负责消息的传输机制,HTTP、Java 消息服务(Java Messaging Service, JMS)和 SMTP 都可以作为服务的消息传输协议,其中 HTTP 使用最广。

(2) 服务通信协议层。服务通信协议层的主要功能是描述并定义服务之间进行消息传递所需的技术标准,常用的标准是 SOAP 和 REST 协议。

(3) 服务描述层。服务描述层主要以一种统一的方式描述服务的接口与消息交换方式,相关的标准是 WSDL。

(4) 服务层。服务层的主要功能是将遗留系统进行包装,并通过发布的 WSDL 接口描述来定位和调用处于服务层的服务。

(5) 业务流程层。业务流程层的主要功能是支持服务发现,服务调用和点到点的服务调用,并将业务流程从服务的底层调用抽象出来。相关的标准是 WSBPEL,有关 WSBPEL 的详细信息见 10.5.4 小节。

(6) 服务注册层。服务注册层的主要功能是使服务提供者能够通过 WSDL 发布服务定义,并支持服务请求者查找所需的服务信息。相关的标准是 UDDI。

2) 服务注册表

服务注册表(service registry)虽然也具有运行时的功能,但主要在 SOA 设计时使用。它提供一个策略执行点(Policy Enforcement Point, PEP),在这个点上,服务可以在 SOA 中注册,从而可以被发现和使用。服务注册表可以包括有关服务和相关构件的配置、依从性和约束文件。从理论上来说,任何帮助服务注册、发现和查找服务合约、元数据和策略的信息库、数据库、目录或其他节点都可以被认为是一个注册表。大多数商用服务注册产品支持服务注册、服务位置和服务绑定功能。

(1) 服务注册。服务注册是指服务提供者向服务注册表发布服务的功能(服务合约),包括服务身份、位置、方法、绑定、配置、方案和策略等描述性属性。使用服务注册表实现 SOA 时,要限制哪些新服务可以向注册表发布、由谁发布以及谁批准和根据什么条件批准等,以便使服务能够有序地注册。

(2) 服务位置。服务位置是帮助服务使用者查询已注册的服务, 寻找符合自身要求的服务。这种查找主要是通过检索服务合约来实现的, 在使用服务注册表实现 SOA 时, 需要规定哪些用户可以访问服务注册表, 以及哪些服务属性可以通过服务注册表进行暴露等。

(3) 服务绑定。服务使用者利用查找到的服务合约来开发代码, 开发的代码将与注册的服务进行绑定, 调用注册的服务, 以及与它们实现互动。可以利用集成的开发环境自动将新开发的服务与不同的新协议、方案和程序间通信所需的其他接口绑定在一起。

3) 企业服务总线

ESB 的概念是从 SOA 发展而来的, 它是一种为进行连接服务提供的标准化的通信基础结构, 基于开放的标准, 为应用提供了一个可靠的、可度量的和高度安全的环境, 并可帮助企业对业务流程进行设计和模拟, 对每个业务流程实施控制和跟踪、分析并改进流程和性能。

在一个复杂的企业计算环境中, 如果服务提供者和服务请求者之间采用直接的端到端的交互, 那么随着企业信息系统的增加和复杂度的提高, 系统之间的关联会逐渐变得非常复杂, 形成一个网状结构, 这将带来昂贵的系统维护费用, 同时也使得 IT 基础设施的复用变得困难重重。ESB 提供了一种基础设施, 消除了服务请求者与服务提供者之间的直接连接, 使得服务请求者与服务提供者之间进一步解耦。

ESB 是由中间件技术实现并支持 SOA 的一组基础架构, 是传统中间件技术与 XML、Web Service 等技术结合的产物, 是在整个企业集成架构下的面向服务的企业应用集成机制。具体来说, ESB 具有以下功能:

(1) 支持异构环境中的服务、消息和基于事件的交互, 并且具有适当的服务级别和可管理性。

(2) 通过使用 ESB, 可以在几乎不更改代码的情况下, 以一种无缝的非侵入方式使现有系统具有全新的服务接口, 并能够在部署环境中支持任何标准。

(3) 充当缓冲器的 ESB (负责在诸多服务之间转换业务逻辑和数据格式) 与服务逻辑相分离, 从而使不同的系统可以同时使用同一个服务, 用不着在系统或数据发生变化时, 改动服务代码。

(4) 在更高的层次, ESB 还提供诸如服务代理和协议转换等功能。允许在多种形式下通过像 HTTP、SOAP 和 JMS 总线的多种传输方式, 主要是以网络服务的形式, 为发表、注册、发现和使用企业服务或界面提供基础设施。

(5) 提供可配置的消息转换翻译机制和基于消息内容的消息路由服务, 传输消息到不同的目的地。

(6) 提供安全和拥有者机制, 以保证消息和服务使用的认证、授权和完整性。

在企业应用集成方面, 与现存的、专有的集成解决方案相比, ESB 具有以下优势:

(1) 扩展的、基于标准的连接。ESB 形成一个基于标准的信息骨架, 使得在系统内部和整个价值链中可以容易地进行异步或同步数据交换。ESB 通过使用 XML、SOAP 和其他标准, 提供了更强大的系统连接性。

(2) 灵活的、服务导向的应用组合。基于 SOA, ESB 使复杂的分布式系统 (包括跨多个应用、系统和防火墙的集成方案) 能够由以前开发测试过的服务组合而成, 使系统具有高度可扩展性。

(3) 提高复用率，降低成本。按照 SOA 方法构建应用，提高了复用率，简化了维护工作，进而减少了系统总体成本。

(4) 减少市场反应时间，提高生产率。ESB 通过构件和服务复用，按照 SOA 的思想简化应用组合，基于标准的通信、转换和连接来实现这些优点。

12.4 软件架构标准

软件架构作为一个概念起源于 1968 年 Edsger Dijkstra 和 20 世纪 70 年代初 David Parnas 的研究，直到 1990 年，软件架构一词才得到广泛使用。计算机科学领域自形成就遇到了与复杂性相关的问题。早期开发人员通过选择正确的数据结构、开发算法和应用关注点的分离来解决复杂问题。伴随着软件密集型系统的出现，学者开始重视软件架构标准的设计。IEEE 1471-2000 是软件架构领域的第一个针对“软件密集型系统架构描述的推荐实现”的正式标准，侧重于软件密集型系统和更通用的软件系统，例如信息系统、嵌入式系统等，在软件开发、操作和演进中起着重要作用。为了达到软件架构标准，降低软件复杂度，以得到论域中较为简单的概念，IEEE 1471-2000 定义了以下软件架构标准要素之间的关系，具体分为 5 个层次：

(1) 任务 (Mission)。任务是指一种使用或操作，是一个系统想要满足一名或多名利益相关者的目标，为达成这个目标，完成相对应的软件架构设计。这是软件架构标准设计的第一步，即需要明确软件的任务是什么，从而根据任务来进行软件架构的设计。任务通常可以是为了更大的利益、为了更高的市场占有率或完善产品系列等，这是软件架构标准中的第一层设计标准。

(2) 环境 (Environment)、系统 (System) 以及架构 (Architecture)。一个完整的系统是由一系列组件组成的，将这些组件组织在一起，相互作用从而完成一个或者一些特殊的功能。在软件开发过程中，系统是如何对其设计、构造、部署和演变有重要影响的组成，包括单个应用程序、传统意义上的系统、子系统、产品线、产品系列、整个企业和其他利益。同时系统不可能单独存在，它总是存在于一个环境之中。在软件架构标准中，将系统范围之外的、对系统有影响、有交互的客观存在定义为环境，有时也称为上下文。并且每一个系统都应该实现一种或多种的软件架构，无论是否有意设计或者自发形成，这是软件架构标准中为了完成任务所进行的第二层设计标准。

(3) 利益相关者 (Stackholder)、架构说明 (Architecture Description) 以及原理 (Rationale)。一个系统拥有多个利益相关者，并且系统架构也需要架构说明来进行描述，从而满足不同利益相关者的利益，从架构描述中可以识别出不同利益相关者的相关利益。同时在设计软件架构时，软件架构工程师需要做出许多取舍以及选择，需要给出选择或者不选择的理由，以及架构设计是如何满足功能性需求和非功能性需求的，这是软件架构标准第三层的设计标准。

(4) 关注 (Concern)、关注点 (Viewpoint)、视图 (View)。一个系统的利益相关者对系统和与其有关的问题进行关注。一个关注点可能由一个或多个利益相关者持有。在整个生命周期中，对系统的需求和要求以及实施和运行的考虑等都可能成为关注点。关注点可以通过多种形式表现出来，例如与一个或多个利益相关者的需求、目标、期望、责任、要求、设计约束、假设、依赖性、质量属性、架构决策、风险或其他与系统有关的问题有关。不同的利益相关者其

关注点不尽相同,有时甚至是矛盾的。站在不同的关注点,就会对系统架构形成不同的视图,视图和关注点是一一对应的关系。并且视图受关注点的制约:关注点确立了构建、解释和分析视图的规范。关注点的制约可以包括语言、符号、模型种类、设计规则,以及建模方法、分析技术和其他对视图的操作。

(5) 关注点库 (Library Viewpoint)、模型 (Model)。关注点库中存放了前人针对各种系统、各种不同的利益相关者、各种不同的关注点总结出来的观察点,方便后人参考使用。

Model, 可直译为模型,它是用来表示视图的方法,即使用图形化的表示,将不同视图下的系统元素组织起来,并将这些元素整体发挥的作用表现出来。

IEEE 1471-2000 作为第一个软件架构设计的标准,侧重于软件架构标准的描述,而不是软件架构的组建过程。首先,IEEE 1471-2000 定义了软件架构元素的内容。其次,它指出软件架构必须存在一个或多个视图,这些视图必须能反映利益相关者的关注点。最后,必须根据每个被认定为关键架构的原理,来演示软件架构工程师考虑的权衡,展示对于软件架构设计的取舍。通过对关键概念和术语的定义,该标准为软件架构的实践提供了基础,并作为未来架构发展的基础。其各个要素的关系如图 12-19 所示。

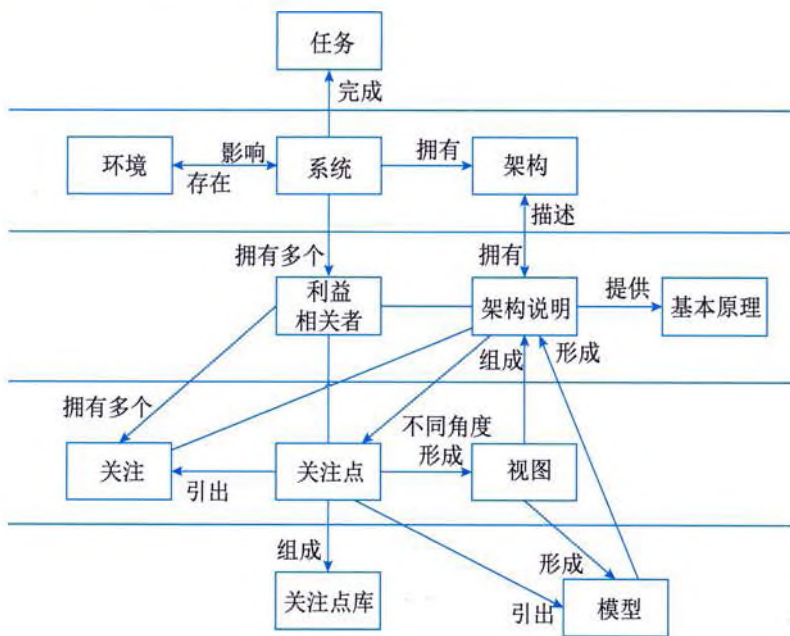


图 12-19 IEEE 1471-2000 软件架构元素及要素间的关系

2006 年,ISO 采纳了 IEEE 1471-2000 作为内部标准,然后 IEEE 与 ISO 一起修订,并经过多次修改,最终发布 ISO/IEC/IEEE 42010: 2022-Software, Systems and Enterprise Architecture Description, 该标准是目前公认的软件架构标准。

12.5 软件架构实现

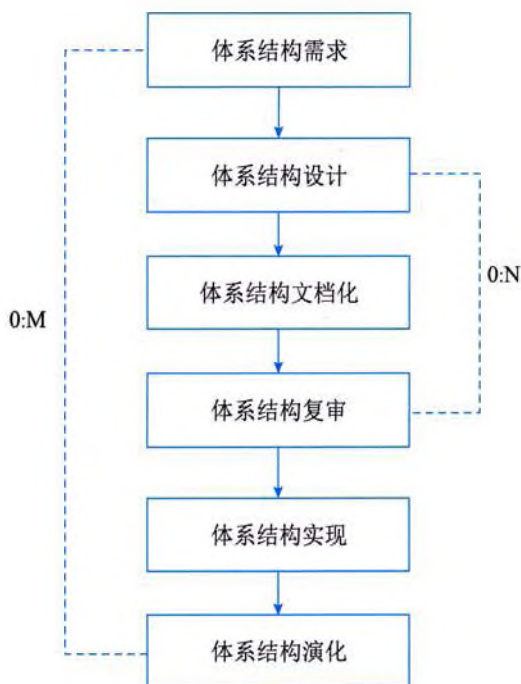


图 12-20 基于架构的软件开发模型

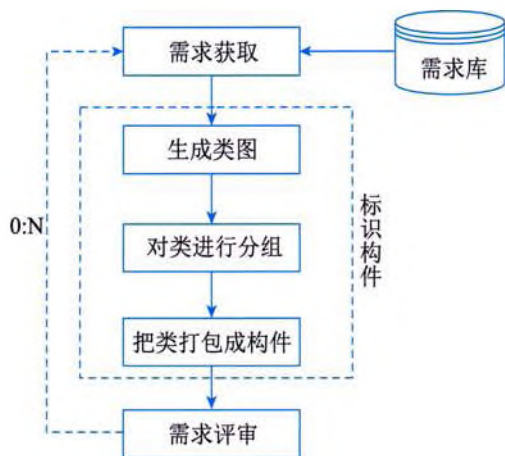


图 12-21 体系结构需求过程

传统的软件开发过程从概念到实现可以划分为若干个阶段，包括问题定义、需求分析、软件设计、软件实现及软件测试等。如果采用传统的软件开发模型，软件体系结构的建立应位于需求分析之后、概要设计之前。

传统软件开发模型存在开发效率不高、不能很好地支持软件重用等缺点。基于架构的软件开发模型（ABSDM）把整个基于体系结构的软件过程划分为体系结构需求、设计、文档化、复审、实现和演化等6个子过程，如图12-20所示。

12.5.1 体系结构需求

需求是指用户对目标软件系统在功能、行为、性能、设计约束等方面的期望。体系结构需求受技术环境和体系结构设计师的经验影响。需求过程主要是获取用户需求，标识系统中所要用到的构件。体系结构需求过程如图12-21所示。如果以前有类似的系统体系结构的需求，我们可以从需求库中取出，加以利用和修改，以节省需求获取的时间，减少重复劳动，提高开发效率。

1. 需求获取

体系结构需求一般来自三个方面，分别是系统的质量目标、系统的商业目标和系统开发人员的商业目标。软件体系结构需求获取过程主要是定义开发人员必须实现的软件功能，使得用户能完成他们的任务，从而满足业务上的功能需求。与此同时，还要获得软件质量属性，满足一些非功能需求。

2. 标识构件

在图12-21中虚线框部分属于标识构件过程，此过程为系统生成初始逻辑结构，包含大致的构件。这一过程又可分为三步来实现。

第一步：生成类图。生成类图的CASE工具有很多，例如Rational Rose 2000能自动生成

类图。

第二步：对类进行分组。在生成的类图基础上，使用一些标准对类进行分组可以大大简化类图结构，使之更清晰。一般地，与其他类隔离的类形成一个组，由概括关联的类组成一个附加组，由聚合或合成关联的类也形成一个附加组。

第三步：把类打包成构件。把在第二步得到的类簇打包成构件，这些构件可以分组合并成更大的构件。

3. 架构需求评审

组织一个由不同代表（如分析人员、客户、设计人员、测试人员）组成的小组，对体系结构需求及相关构件进行仔细的审查。审查的主要内容包括所获取的需求是否真实反映了用户的要求，类的分组是否合理，构件合并是否合理等。必要时，可以在“需求获取—标识构件—需求评审”之间进行迭代。

12.5.2 体系结构设计

体系结构需求用来激发和调整设计决策，不同的视图被用来表达与质量目标有关的信息。体系结构设计是一个迭代过程，如果要开发的系统能够从已有的系统中导出大部分，则可以使用已有系统的设计过程。软件体系结构设计过程如图 12-22 所示。

1. 提出体系结构模型

在建立体系结构的初期，选择一个合适的体系结构风格是首要的。在这个风格基础上，开发人员通过体系结构模型，可以获得关于体系结构属性的理解。此时，虽然这个模型是理想化的（其中的某些部分可能错误地表示了应用的特征），但是，该模型为将来的实现和演化过程建立了目标。

2. 映射构件

把在体系结构需求阶段已标识的构件映射到体系结构中，将产生一个中间结构，这个中间结构只包含那些能明确适合体系结构模型的构件。

3. 分析构件间的相互作用

为了把所有已标识的构件集成到体系结构中，必须认真分析这些构件的相互作用和关系。

4. 产生体系结构

一旦决定了关键的构件之间的关系和相互作用，就可以在第 2 阶段得到的中间结构的基础上进行精化。

5. 设计评审

一旦设计了软件体系结构，必须邀请独立于系统开发的外部人员对体系结构进行评审。

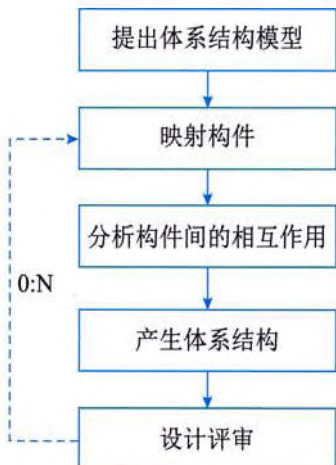


图 12-22 软件体系结构设计过程

12.5.3 体系结构文档化

绝大多数的体系结构都是抽象的，由一些概念上的构件组成。例如，层的概念在任何程序设计语言中都不存在。因此，要让系统分析员和程序员去实现体系结构，还必须要把体系结构进行文档化。在系统演化的每一个阶段，文档都是系统设计与开发人员的通信媒介，是为验证体系结构设计和提炼或修改这些设计（必要时）所执行预先分析的基础。

体系结构文档化过程的主要输出结果是体系结构规格说明和测试体系结构需求的质量设计说明书这两个文档。这些文档是需求模型构件的精确的形式化的描述，作为用户和开发者之间的一个协约。软件体系结构文档的要求与软件开发项目中的其他文档是类似的。文档的完整性和质量是软件体系结构成功的关键因素。文档要从使用者的角度进行编写，必须分发给所有与系统有关的开发人员，且必须保证开发者手上的文档是最新的。

12.5.4 体系结构复审

从图 12-20 中可以看出，体系结构设计、体系结构文档化和体系结构复审是一个迭代过程。从这个角度来说，在分析一个主版本的软件体系结构之后，要安排一次由外部人员（用户代表和领域专家）参加的复审。

鉴于体系结构文档标准化，以及风险识别的现实情况，通常我们根据架构设计，搭建一个可运行的最小化系统用于评估和测试体系架构是否满足需要，是否存在可识别的技术和协作风险。

复审的目的是标识潜在的风险，及早发现体系结构设计中的缺陷和错误，包括体系结构能否满足需求、质量需求是否在设计中得到体现、层次是否清晰、构件的划分是否合理、文档表达是否明确、构件的设计是否满足功能与性能的要求等。

12.5.5 体系结构实现

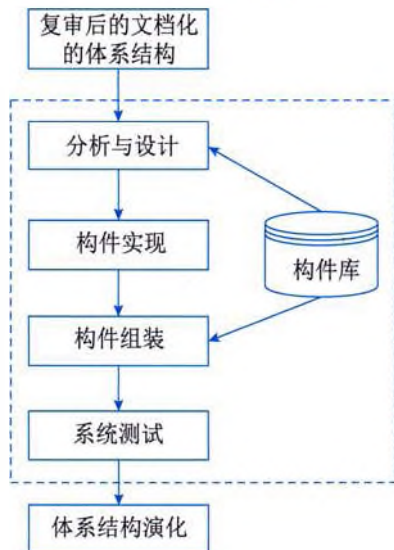


图 12-23 体系结构实现过程

所谓“实现”就是要用实体来显示出一个软件体系结构，即要符合体系结构所描述的结构设计决策，分割成规定的构件，按规定方式互相交互。体系结构的实现过程如图 12-23 所示。

图 12-23 中的虚线框部分是体系结构的实现过程。整个实现过程是以复审后的文档化的体系结构说明书为基础的，每个构件必须满足软件体系结构中说明的对其他构件的责任。这些决定即实现的约束是在系统级或项目范围内给出的，每个构件上工作的实现者是看不见的。

在体系结构说明书中，已经定义了系统中的构件与构件之间的关系。因为在体系结构层次上，构件接口约束对外唯一地代表了构件，所以可以从构件库中查找符合接口约束的构件，必要时开发新的满足要求的构件。然后，按照设计提供的结构，通过组装支持工具把这些构件的实现

体组装起来,完成整个软件系统的连接与合成。

最后一步是测试,包括单个构件的功能性测试和被组装应用的整体功能和性能测试。

12.5.6 体系结构的演化

在构件开发过程中,用户的需求可能还有变动。在软件开发完毕,正常运行后,由一个单位移植到另一个单位,需求也会发生变化。在这两种情况下,就必须相应地修改软件体系结构,以适应新的变化了的软件需求。体系结构演化过程如图12-24所示。

体系结构演化是使用系统演化步骤去修改应用,以满足新的需求。主要包括以下6个步骤。

1. 需求变化归类

首先必须对用户需求的变化进行归类,使变化的需求与已有构件对应。对找不到对应构件的变动,要特别做好标记,在后续工作中,将创建新的构件,以对应这部分变化的需求。

2. 制订体系结构演化计划

在改变原有结构之前,开发组织必须制订一个周密的体系结构演化计划,作为后续演化开发工作的指南。

3. 修改、增加或删除构件

在演化计划的基础上,开发人员可根据在第1步得到的需求变动的归类情况,决定是否修改或删除存在的构件、增加新构件。最后,对修改和增加的构件进行功能性测试。

4. 更新构件的相互作用

随着构件的增加、删除和修改,构件之间的控制流必须得到更新。

5. 构件组装与测试

通过组装支持工具把这些构件的实现体组装起来,完成整个软件系统的连接与合成,形成新的体系结构。然后对组装后的系统整体功能和性能进行测试。

6. 技术评审

对以上步骤进行确认,进行技术评审。评审组装后的体系结构是否反映需求变动,符合用户需求。如果不符合,则需要第2~6步之间进行迭代。

在原来系统上所作的所有修改必须集成到原来的体系结构中,完成一次演化过程。

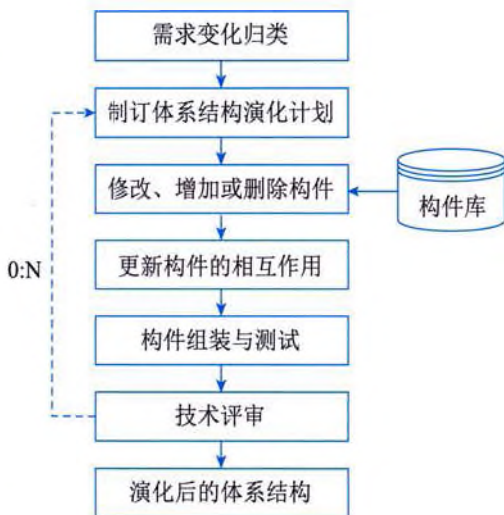


图 12-24 体系结构演化过程

12.6 软件架构质量属性

软件系统属性包括功能属性和质量属性，软件架构重点关注的是质量属性。架构的基本需求是在满足功能属性的前提下，关注软件系统质量属性。为了精确、定量地表达系统的质量属性，通常会采用质量属性场景的方式进行描述。

在确定软件系统架构，精确描述质量属性场景后，就需要对系统架构进行评估。软件架构评估是在对架构分析、评估的基础上，对架构策略的选取进行决策。它也可以灵活地运用于软件架构评审等工作。

12.6.1 质量属性概念

软件系统的质量就是“软件系统与明确地和隐含地定义的需求相一致的程度”。更具体地说，软件系统质量是软件与明确叙述的功能和性能需求文档中明确描述的开发标准以及任何专业开发的软件产品都应该具有的隐含特征相一致的程度。根据 GB/T 25000.10—2016 的定义，从管理角度对软件系统质量进行度量，可将影响软件质量的主要因素划分为 6 种维度特性：功能性、可靠性、易用性、效率、维护性与可移植性。其中，功能性包括适合性、准确性、互操作性、依从性和安全性；可靠性包括容错性、易恢复性和成熟性；易用性包括易学性、易理解性和易操作性；效率包括资源特性和时间特性；维护性包括可测试性、可修改性、稳定性和易分析性；可移植性包括适应性、易安装性、一致性和可替换性。

软件系统质量属性（Quality Attribute）是一个系统的可测量或者可测试的属性，用来描述系统满足利益相关者（Stakeholders）需求的程度。基于软件系统的生命周期，可以将软件系统的质量属性分为开发期质量属性和运行期质量属性两部分。

1. 开发期质量属性

开发期质量属性主要指在软件开发阶段所关注的质量属性，主要包含以下 6 个方面：

- （1）易理解性：指设计被开发人员理解的难易程度。
- （2）可扩展性：指软件因适应新需求或需求变化而增加新功能的能力，也称为灵活性。
- （3）可重用性：指重用软件系统或某一部分的难易程度。
- （4）可测试性：指对软件测试以证明其满足需求规范的难易程度。
- （5）可维护性：当需要修改缺陷、增加功能、提高质量属性时，识别修改点并实施修改的难易程度。
- （6）可移植性：将软件系统从一个运行环境转移到另一个不同的运行环境的难易程度。

2. 运行期质量属性

运行期质量属性主要指在软件运行阶段所关注的质量属性，主要包含以下 7 个方面：

- （1）性能：指软件系统及时提供相应服务的能力，如速度、吞吐量和容量等的要求。
- （2）安全性：指软件系统同时兼顾向合法用户提供服务，以及阻止非授权使用的能力。
- （3）可伸缩性：指当用户数和数据量增加时，软件系统维持高服务质量的能力。例如，通过增加服务器来提高能力。

(4) 互操作性：指本软件系统与其他系统交换数据和相互调用服务的难易程度。

(5) 可靠性：指软件系统在一定的时间内持续无故障运行的能力。

(6) 可用性：指系统在一定时间内正常工作的时间所占的比例。可用性会受到系统错误、恶意攻击和高负载等问题的影响。

(7) 健壮性：指软件系统在非正常情况（如用户进行了非法操作、相关的软硬件系统发生了故障等）下仍能够正常运行的能力，也称为鲁棒性或容错性。

12.6.2 面向架构评估的质量属性

为了评价一个软件系统，特别是软件系统的架构，需要进行架构评估。在架构评估过程中，评估人员所关注的是系统的质量属性。评估方法所普遍关注的质量属性有以下几种。

1. 性能

性能（Performance）是指系统的响应能力，即要经过多长时间才能对某个事件做出响应，或者在某段时间内系统所能处理的事件的个数。经常用单位时间内所处理事务的数量或系统完成某个事务处理所需的时间来对性能进行定量的表示。性能测试经常要使用基准测试程序。

2. 可靠性

可靠性（Reliability）是软件系统在应用或系统错误面前，在意外或错误使用的情况下维持软件系统的功能特性的基本能力。可靠性是最重要的软件特性，通常用来衡量在规定的条件和时间内，软件完成规定功能的能力。可靠性通常用平均无故障时间（MTTF）和平均故障间隔时间（MTBF）来衡量。在失效率为常数和修复时间很短的情况下，MTTF 和 MTBF 几乎相等。可靠性可以分为两个方面。

(1) 容错。其目的是在错误发生时确保系统正确的行为，并进行内部“修复”。例如，在一个分布式软件系统中失去了一个与远程构件的连接，接下来恢复了连接。在修复这样的错误之后，软件系统可以重新或重复执行进程间的操作直到错误再次发生。

(2) 健壮性。这里说的健壮性是保护应用程序不受错误使用和错误输入的影响，在发生意外错误事件时确保应用系统处于预先定义好的状态。值得注意的是，和容错相比，健壮性并不是指在错误发生时软件可以继续运行，它只能保证软件按照某种已经定义好的方式终止执行。软件架构对软件系统的可靠性有巨大的影响。例如，软件架构设计上通过在应用程序内部采用冗余机制，或集成监控构件和异常处理，以提升系统可靠性。

3. 可用性

可用性（Availability）是系统能够正常运行的时间比例，经常用两次故障之间的时间长度或在出现故障时系统能够恢复正常的速度来表示。

4. 安全性

安全性（Security）是指系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力。安全性可根据系统可能受到的安全威胁类型来分类。安全性又可划分为机密性、完整性、不可否认性及可控性等特性。其中，机密性保证信息不泄露给未授权的用户、

实体或过程；完整性保证信息的完整和准确，防止信息被非法修改；不可否认性是指信息交换的双方不能否认其在交换过程中发送信息或接收信息的行为；可控性保证对信息的传播及内容具有控制的能力，防止为非法者所用。

5. 可修改性

可修改性（Modifiability）是指能够快速地对系统以较高的性价比进行变更的能力。通常以某些具体的变更为基准，通过考察这些变更的代价衡量可修改性。可修改性包含以下4个方面。

（1）可维护性（Maintainability）。这主要体现在问题的修复上：在错误发生后“修复”软件系统。可维护性好的软件架构往往能做局部性的修改，并能使这种修改对其他构件的负面影响最小化。

（2）可扩展性（Extendibility）。这一点关注的是使用新特性来扩展软件系统，以及使用改进版本方式替换构件并删除不需要或不必要的特性和构件。为了实现可扩展性，软件系统需要松散耦合的构件。其目标是实现一种架构，能使开发人员在不影响构件客户的情况下替换构件。支持把新构件集成到现有的架构中也是必要的。

（3）结构重组（Reassemble）。这一点处理的是重新组织软件系统的构件及构件间的关系，例如通过将构件移动到一个不同的子系统来改变它的位置。为了支持结构重组，软件系统需要精心设计构件之间的关系。理想情况下，它们允许开发人员在不影响实现的主体部分的情况下灵活地配置构件。

（4）可移植性（Portability）。可移植性使软件系统适用于多种硬件平台、用户界面、操作系统、编程语言或编译器。为了实现可移植，需要按照硬件、软件无关的方式组织软件系统。可移植性是系统能够在不同计算环境下运行的能力，这些环境可能是硬件、软件，也可能是两者的结合。如果移植到新的系统需要做适当更改，则该可移植性就是一种特殊的可修改性。

6. 功能性

功能性（Functionality）是系统完成所期望工作的能力。一项任务的完成需要系统中许多或大多数构件的相互协作。

7. 可变性

可变性（Changeability）是指架构经扩充或变更而成为新架构的能力。这种新架构应该符合预先定义的规则，在某些具体方面不同于原有的架构。当要将某个架构作为一系列相关产品（例如，软件产品线）的基础时，可变性是很重要的。

8. 互操作性

作为系统组成部分的软件不是独立存在的，通常与其他系统或自身环境相互作用。为了支持互操作性，软件架构必须为外部可视的功能特性和数据结构提供精心设计的软件入口。程序和用其他编程语言编写的软件系统的交互作用就是互操作性的问题，这种互操作性也影响应用的软件架构。

12.6.3 质量属性场景描述

为了精确描述软件系统的质量属性，通常采用质量属性场景（Quality Attribute Scenario）作为描述质量属性的手段。质量属性场景是一个具体的质量属性需求，是利益相关者与系统的交互的简短陈述。

质量属性场景是一种面向特定质量属性的需求。它由6部分组成：

刺激源（Source）：指某个生成该刺激的实体（人、计算机系统或者任何其他刺激器）。

刺激（Stimulus）：该刺激是当刺激到达系统时需要考虑的条件。

环境（Environment）：该刺激在某些条件下发生。当激励/刺激产生时，系统可能处于过载或者运行状态，也可能是其他情况。

制品（Artifact）：某个制品被激励。这可能是整个系统，也可能是系统的一部分。

响应（Response）：该响应是在激励到达后所采取的行动。

响应度量（Measurement）：当响应发生时，应当能以某种方式对其进行度量，以对需求进行测试。

质量属性场景主要关注可用性、可修改性、性能、可测试性、易用性和安全性等6类质量属性，下面分别列表进行介绍。

1. 可用性质量属性场景

可用性质量属性场景所关注的方面包括系统故障发生的频率、出现故障时会发生什么情况、允许系统有多长时间正常运行、什么时候可以安全地出现故障、如何防止故障的发生以及发生故障时要求进行哪种通知。表12-1给出了详细描述。

表 12-1 可用性质量属性场景描述

场景要素	可能的情况
刺激源	系统内部、系统外部
刺激	疏忽、错误、崩溃、时间
环境	正常操作、降级模式
制品	系统处理器、通信信道、持久存储器、进程
响应	系统应该检测事件，并进行如下一项或多项活动： ● 将其记录下来通知适当的各方，包括用户和其他系统 ● 根据已定义的规则禁止导致错误或故障的事件源 ● 在一段预先指定的时间间隔内不可用，时间间隔取决于系统的关键程度在正常或降级模式下运行
响应度量	系统必须可用的时间间隔、可用时间、系统可以在降级模式下运行的时间间隔、故障修复时间

2. 可修改性质量属性场景

可修改性质量属性场景主要关注系统在改变功能、质量属性时需要付出的成本和难度，可

修改性质量属性场景可能发生在系统设计、编译、构建、运行等多种情况和环境下，表 12-2 给出了详细描述。

表 12-2 可修改性质量属性场景描述

场景要素	可能的情况
刺激源	最终用户、开发人员、系统管理员
刺激	希望增加、删除、修改、改变功能、质量属性、容量等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	查找架构中需要修改的位置，进行修改且不会影响其他功能，对所做的修改进行测试，部署所做的修改
响应度量	根据所影响元素的数量和修改需求难度等，度量修改该元素的人力成本、修改所需的时间、修改的资金成本

3. 性能质量属性场景

性能质量属性场景主要关注系统的响应速度，可以通过效率、响应时间、吞吐量、负载来客观评价性能的好坏，表 12-3 给出了详细描述。

表 12-3 性能质量属性场景描述

场景要素	可能的情况
刺激源	用户请求，其他系统触发等
刺激	定期事件到达；随机事件到达；偶然事件到达
环境	正常模式；超载（overload）模式
制品	系统
响应	处理刺激；改变服务级别
响应度量	等待时间、期限、吞吐量、抖动、缺失率、数据丢失率

4. 可测试性质量属性场景

可测试性质量属性场景主要关注系统测试过程中的效率，发现系统缺陷或故障的难易程度等，表 12-4 给出了详细描述。

表 12-4 可测试性质量属性场景描述

场景要素	可能的情况
刺激源	开发人员、增量开发人员、系统验证人员、客户验收测试人员、系统用户
刺激	已完成的分析、架构、设计、类和子系统集成；所交付的系统
环境	设计时、开发时、编译时、部署时
制品	设计、代码段、完整的应用
响应	提供对状态值的访问、提供所计算的值、准备测试环境
响应度量	已执行的可执行语句的百分比、如果存在缺陷出现故障的概率、执行测试的时间、测试中最长依赖的长度、准备测试环境的时间

5. 易用性质量属性场景

易用性质量属性场景主要关注用户在使用系统时的容易程度，包括系统的学习曲线、完成操作的效率、对系统使用过程的满意程度等，表 12-5 给出了详细描述。

表 12-5 易用性质量属性场景描述

场景要素	可能的情况
刺激源	最终用户
刺激	想要学习系统特性、有效使用系统、使错误的影响最低、适配系统、对系统满意
环境	系统运行时或配置时
制品	系统
响应	(1) 系统提供以下一个或多个响应来支持“学习系统特性”： 帮助系统与环境联系紧密；界面为用户所熟悉；在不熟悉的环境中，界面是可以使用的 (2) 系统提供以下一个或多个响应来支持“有效使用系统”： 数据和（或）命令的聚合；已输入的数据和（或）命令的重用；支持在界面中的有效导航；具有一致操作的不同视图；全面搜索；多个同时进行的活动 (3) 系统提供以下一个或多个响应来“使错误的影响最低”： 撤销；取消；从系统故障中恢复；识别并纠正用户错误；检索忘记的密码；验证系统资源 (4) 系统提供以下一个或多个响应来“适配系统”： 定制能力；国际化 (5) 系统提供以下一个或多个响应来使客户“对系统满意”： 显示系统状态；与客户的节奏合拍
响应度量	任务时间、错误数量、解决问题的数量、用户满意度、用户知识的获得、成功操作在总操作中所占的比例、损失的时间 / 丢失的数据量

6. 安全性质量属性场景

安全性质量属性场景主要关注系统在安全性方面的要素，衡量系统在向合法用户提供服务的同时，阻止非授权用户使用的能力，表 12-6 给出了详细描述。

表 12-6 安全性质量属性场景描述

场景要素	可能的情况
刺激源	正确识别、非正确识别或身份未知的个人或系统（来自内部或外部）；经过了授权 / 未授权访问了有限的资源或大量资源
刺激	试图显示数据、改变 / 删除数据、访问系统服务、降低系统服务的可用性
环境	在线或离线、联网或断网、连接有防火墙或者直接连到了网络
制品	系统服务、系统中的数据
响应	对用户身份进行认证；隐藏用户的身份；阻止对数据或服务的访问；允许访问数据或服务；授予或收回对访问数据或服务的许可；根据身份记录访问 / 修改或试图访问 / 修改数据 / 服务；以一种不可读的格式存储数据；识别无法解释的对服务的高需求；通知用户或另外一个系统，并限制服务的可用性
响应度量	用成功的概率表示，避开安全防范措施所需要的时间 / 努力 / 资源；检测到攻击的可能性；确定攻击或访问 / 修改数据或服务的个人的可能性；在拒绝服务攻击的情况下仍然获得服务的百分比；恢复数据 / 服务；被破坏的数据 / 服务和 / 或被拒绝的合法访问的范围